脱壳学习（二）- 反“反调试”篇

2023-05-21 802 发布于天津

版权 ···

今天这篇主要讲述**常见的反调试技术和如何绕过反调试的实例**。调试无论是在脱壳还是PWN及逆向中都有着很重要的意义，反调试被程序作者用来保护程序不被调试，以此来保护自己的秘密。不过逆行分析人员也有自己的破解反调试的方法，就是“反反调试”。

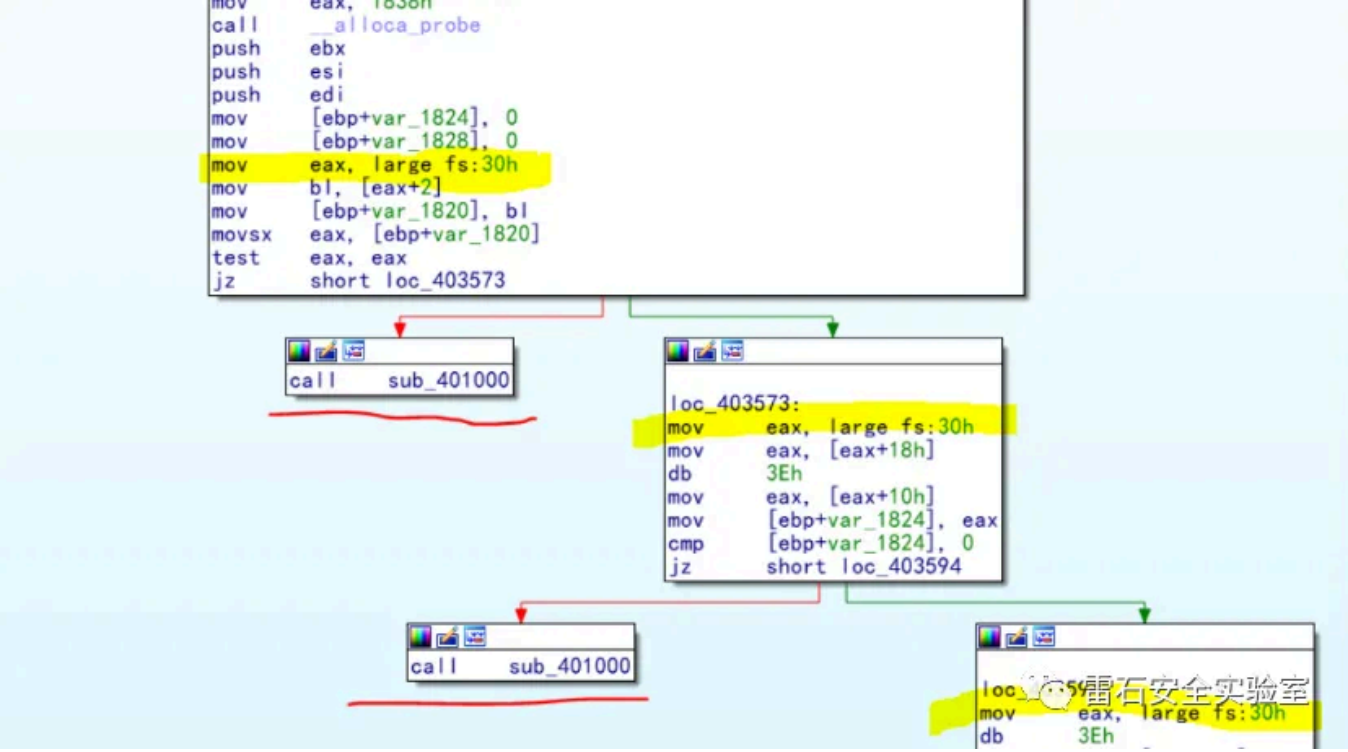
常见反调试

以下是我们经常会遇到到的一些反调试技术，为了好记一点，总结分为以下三类：**通过调试的痕迹去识别是否正在被调试、识别调试器行为、干扰调试器功能**。

1.调试痕迹识别

WindowsAPI
手动检测数据结构
系统痕迹检测

这里最常见得就是**手动检测数据结构**，也就是检测PEB（PEB表存放进程信息）表BeingDebugged，ProcessHeap，NTGlobalFlag属性，这里有个小知识，32位程序PEB得地址用的是fs:[30]，64位程序的地址为gs:[60]，位于用户地址空间，进程环境块的地址对于每个进程来说是固定的。如果发现程序中出现引用PEB的地址然后出现程序退出的地址比如sub_401000，那大概率是进行了反调试。



绕过：fs段寄存器偏移0x30h，用ctrl+G可以找到PEB。第一个fs:[30h]+2指向了PEB的BeingDubgged，这是检验反调试的一种技术。BeingDubgged=1的时候就是被调试，这里是第一处反调试。然后我们通常会在fs:[30h]+2把01修改为00或使用插件：PhantOM,勾选hide from PEB。

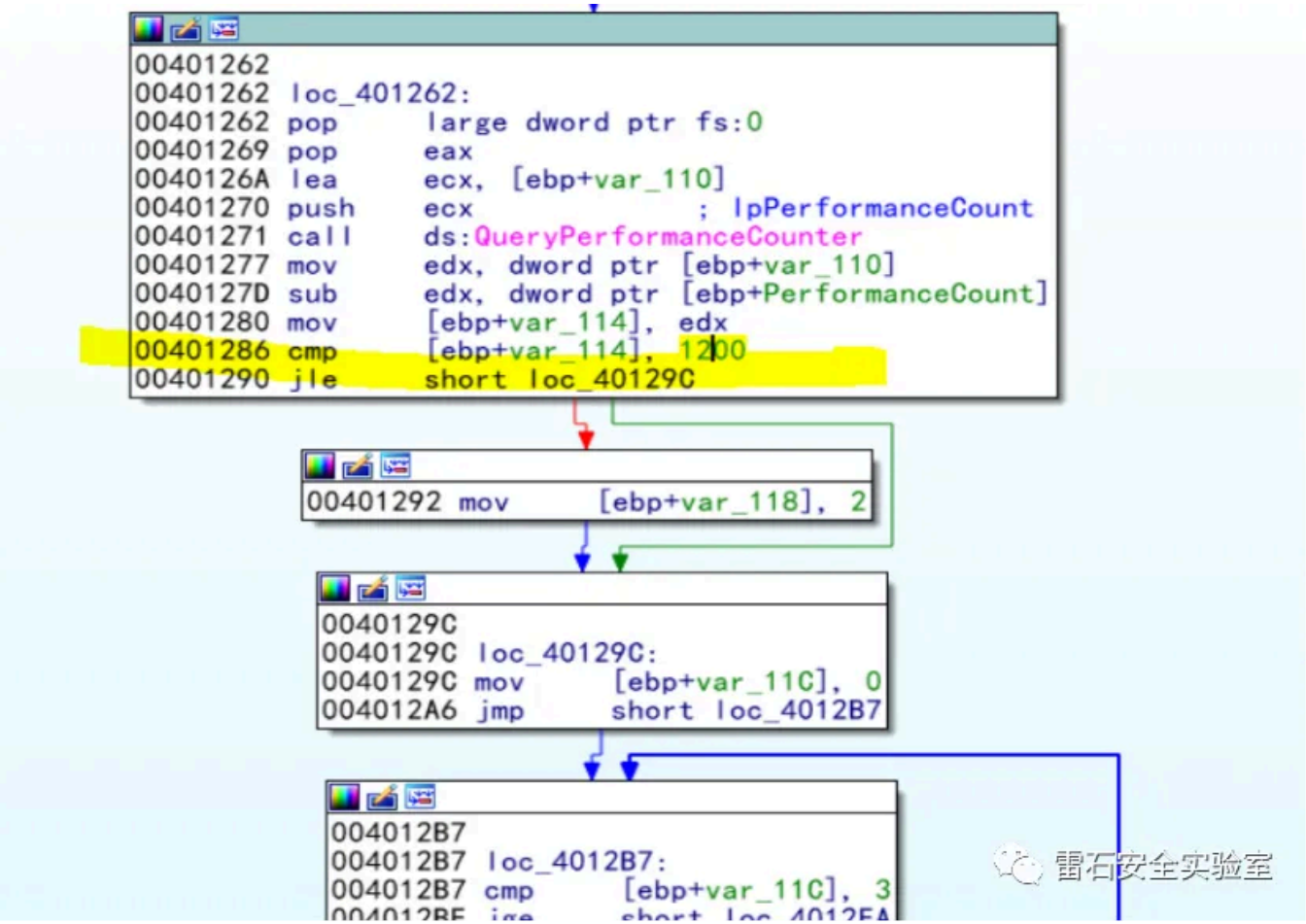
[fs:[30h]+18]+10,ProcessHeap也是用来测试反调试的的标志。fs:30h+68，NtGlobalFlag反调试。同样是BeingDubgged原理，通过修改调试标志来绕过。

2.识别调试器的行为

INT扫描
执行代码校验和检查
时钟监测

判断父进程是否是explorer.exe

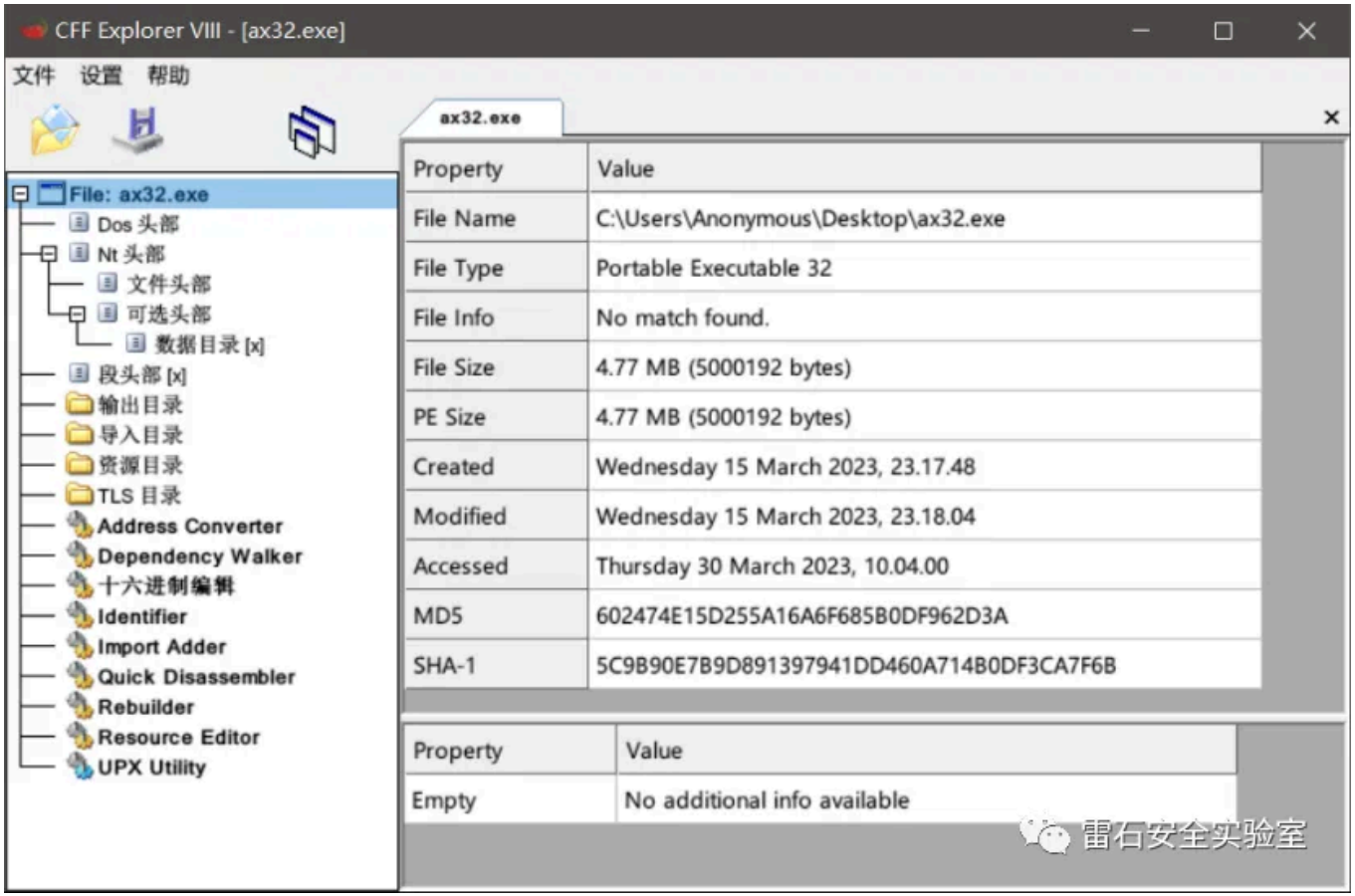
这里常用就是**时钟监测，计算时间差值，反调试**。如果时间过长，判断文件当前正在被调试，就删除文件。我们在OD里找到这个时钟的地方，QueryPerformanceCounter，GetTickCount，
使用nop（汇编代码无操作空指令）就可以绕过，其实原理就是破坏他程序监测反调试的功能。其他几种也是同样的道理，检测都是又程序编写，代码控制，我们只要破坏掉就可以了。



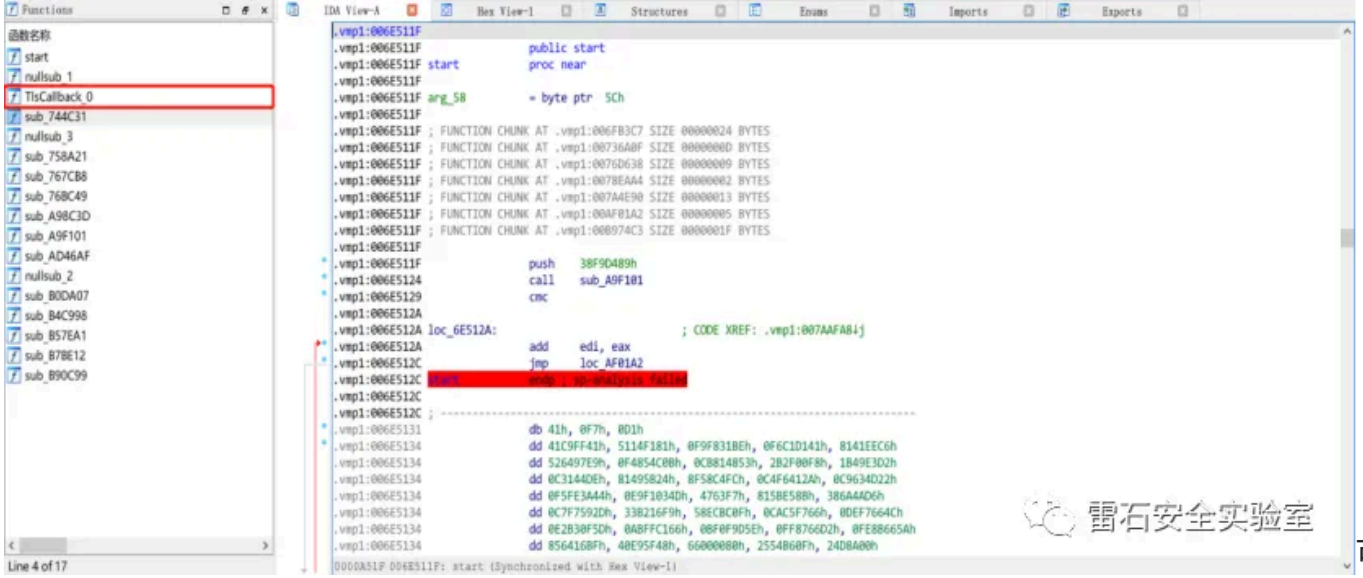
3.干扰调试器的功能

使用TLS回调
使用异常
插入中断

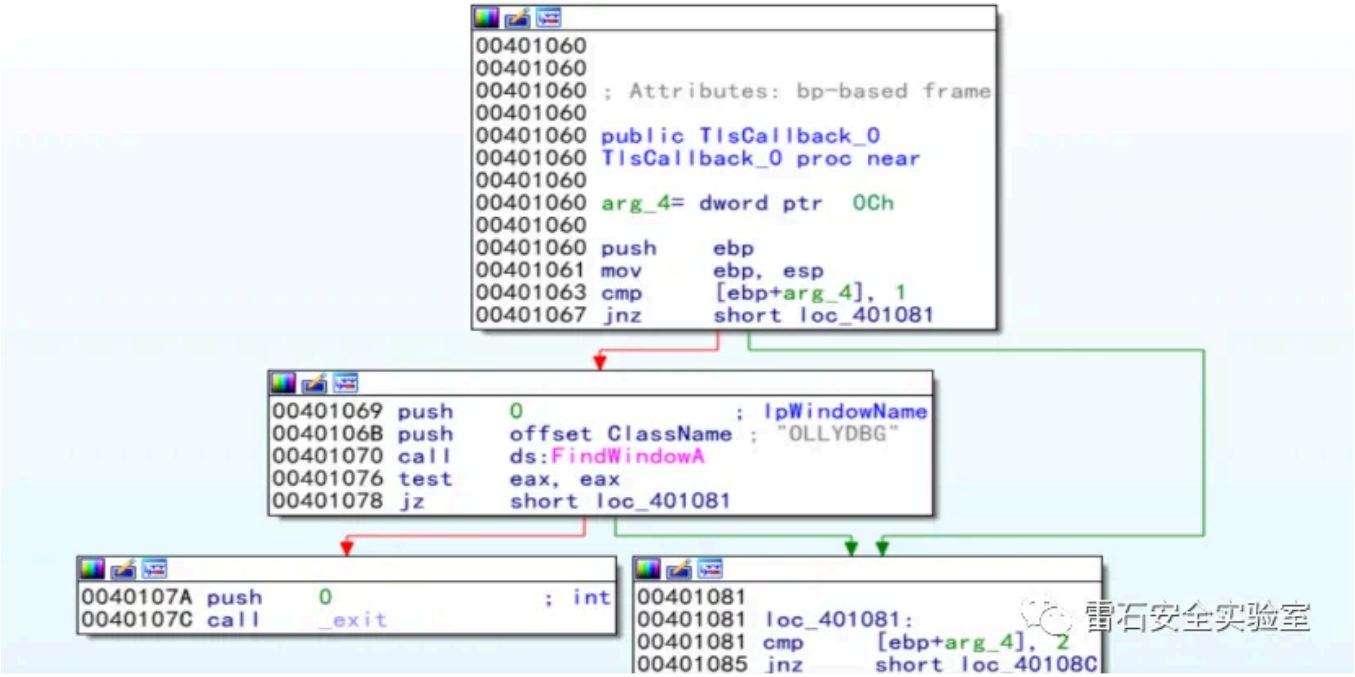
这里最常用的也是**TLS回调**，因为现在OD的插件越来越牛，新版的OD看不出有什么区别，最简单的方法，扔到CFF去看一下目录结构，可以很明确的看到TLS目录(全保护)



然后我们打开IDA可以看到调用的函数

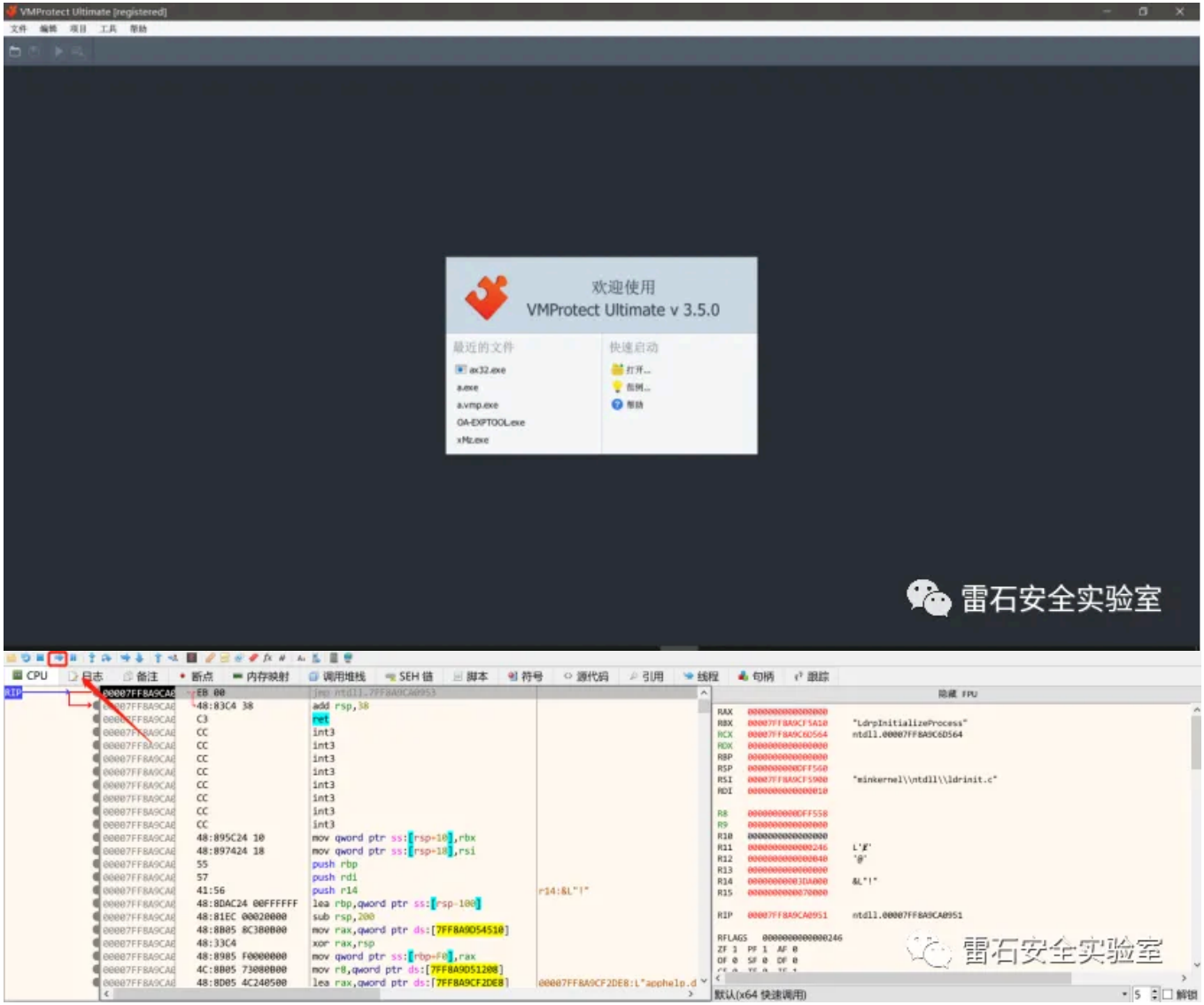


可以看到字符如果当前窗口类名为OLLYDBG“，就退出程序。这个是最简单的干扰，实际上现在常用的OD都已经会隐藏自己的窗口类名了，插件的功能越来越多，TLS回调对于反调试的干扰越来越弱。

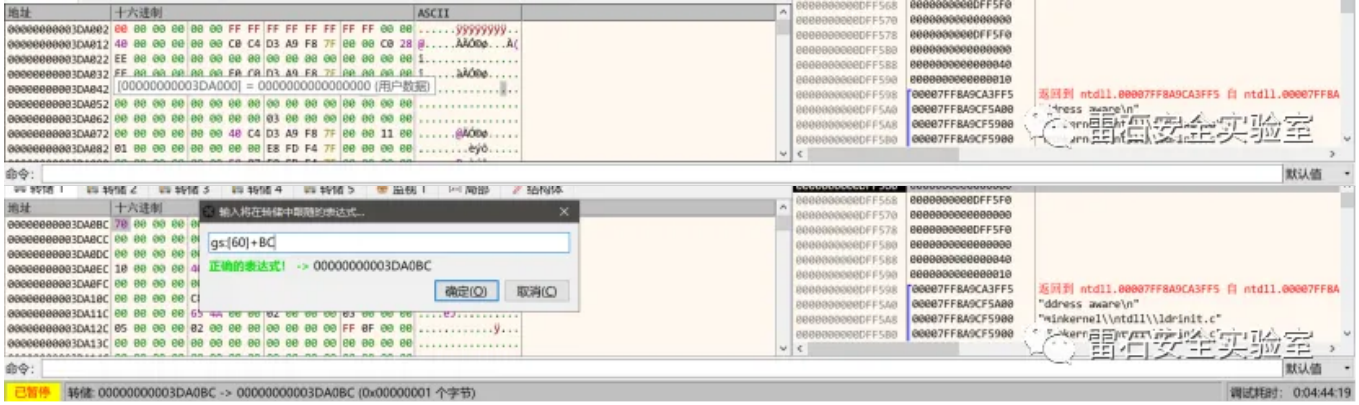


实例

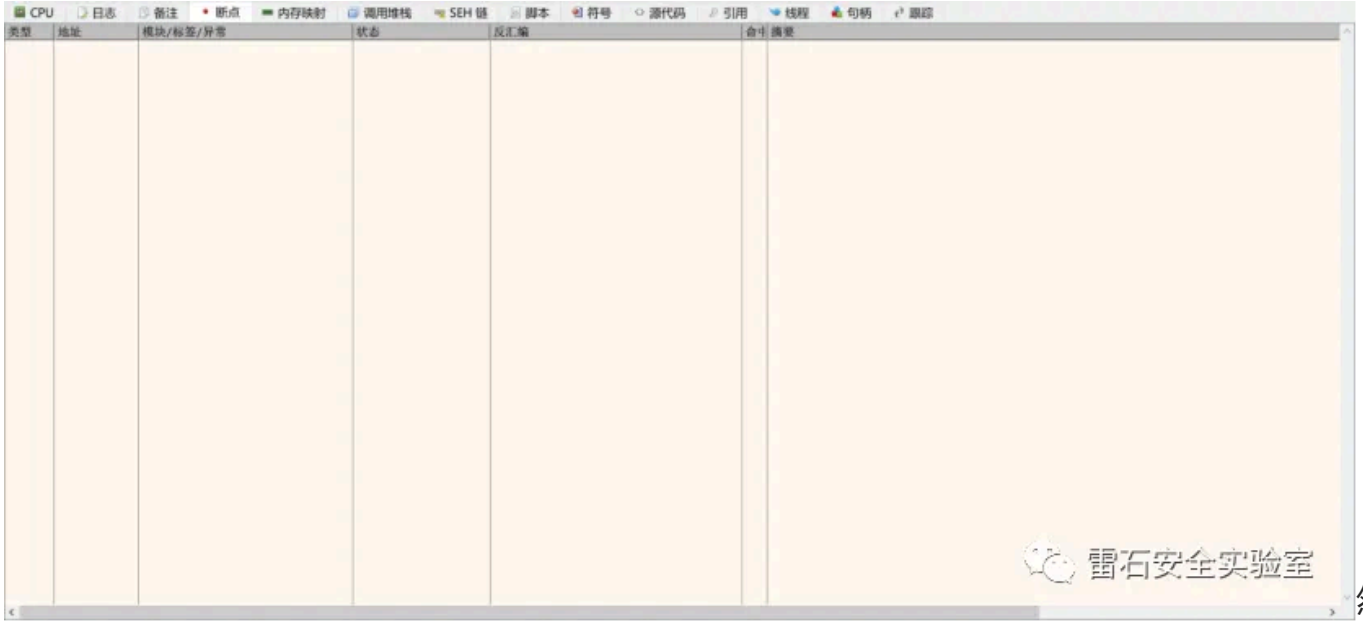
以上就是比较常见的反调试。下面就是一个简单的小实例。这里用**VMProtect3.5进行全保护加壳**。代码就很简单，c语言helloword！。



先点击运行，VMP3.5的全保护首先就是要过掉BeingDebugger和NTGLObalflags，我这里用的程序是64位，所以PEB地址为gs:[60],BeingDebugger和NTGLObalflags的地址PEB+2 -> BeingDebugger，PEB + BC ->NTGLObalflags。



删除所有断点：

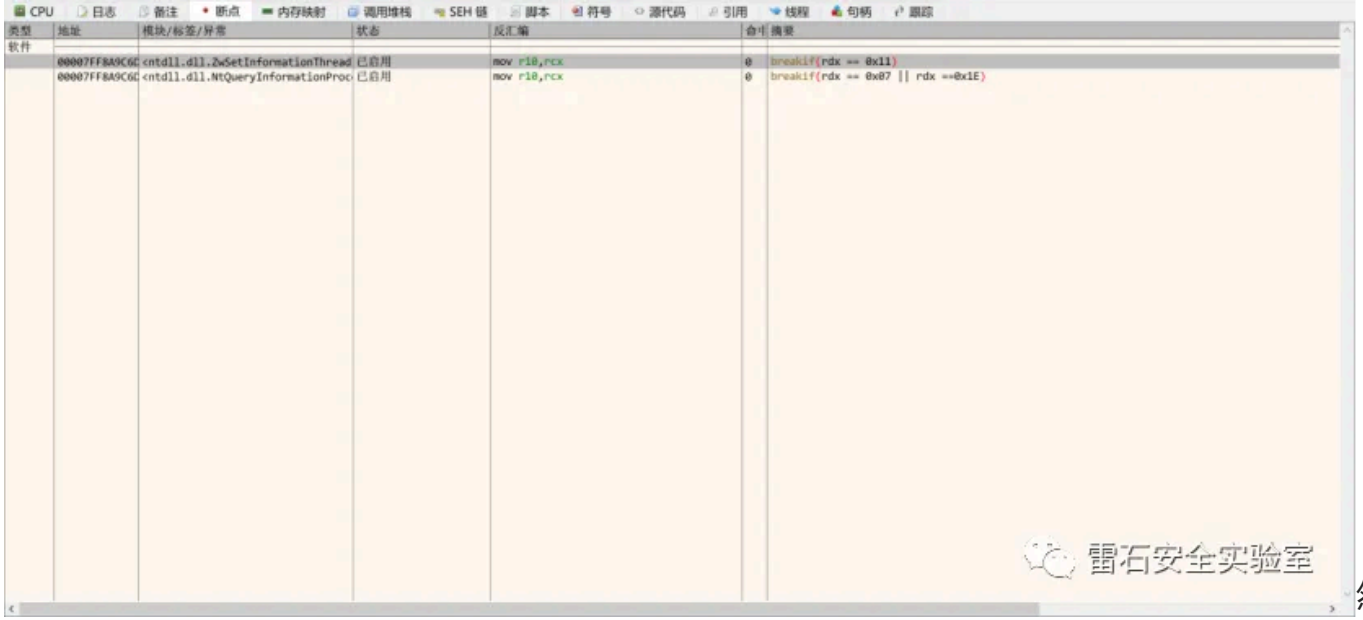


然后给这两个函数下断点 设置条件断点

NtQueryInformationProcess:函数就可以获取调试端口。若处于调试状态，第三个参数会被置为0xFFFFFFFF(-1)；若处于非调试状态,第三个参数值会被设置为0。

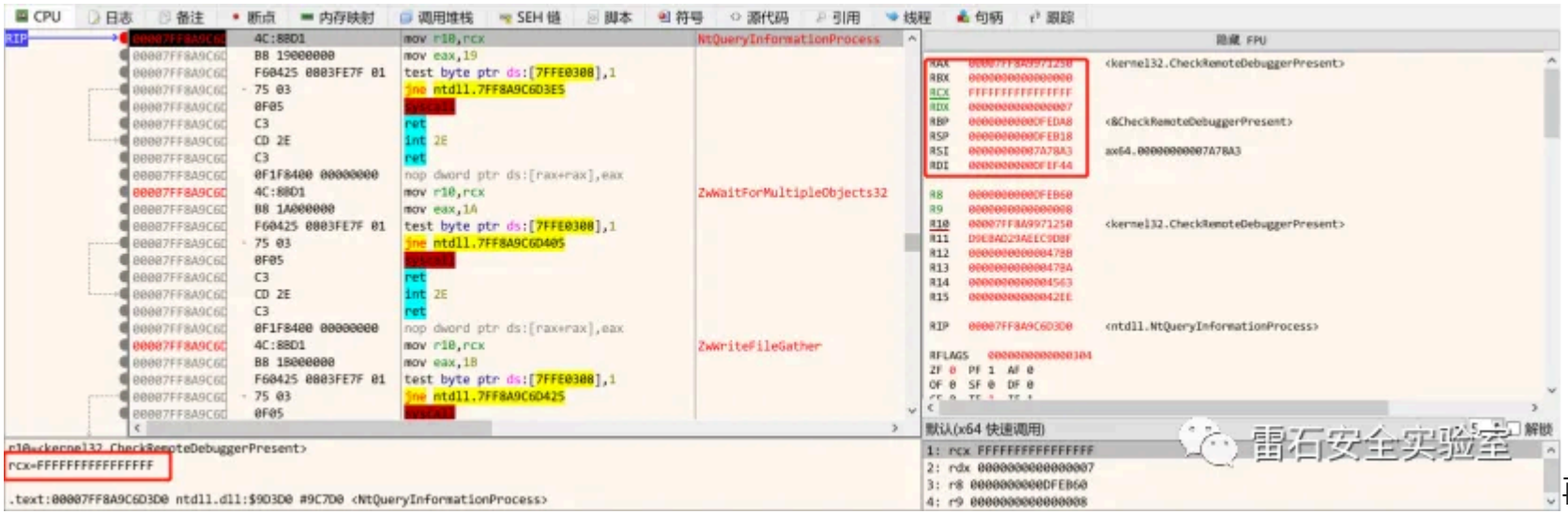
设定条件断点rdx == 0x07 || rdx ==0x1E

NtSetInformationThread:使用Windows的一个未公开函数的方法，你可以在当前线程里调用NtSetInformationThread，调用这个函数时，如果在第二个参数里指定0x11这个值（意思是ThreadHideFromDebugger），等于告诉操作系统，将所有附加的调试器统统取消掉。设定条件断点rdx == 0x11



然后运行 执行顺序可能不一样 但处理方法一样 我这里先断在了

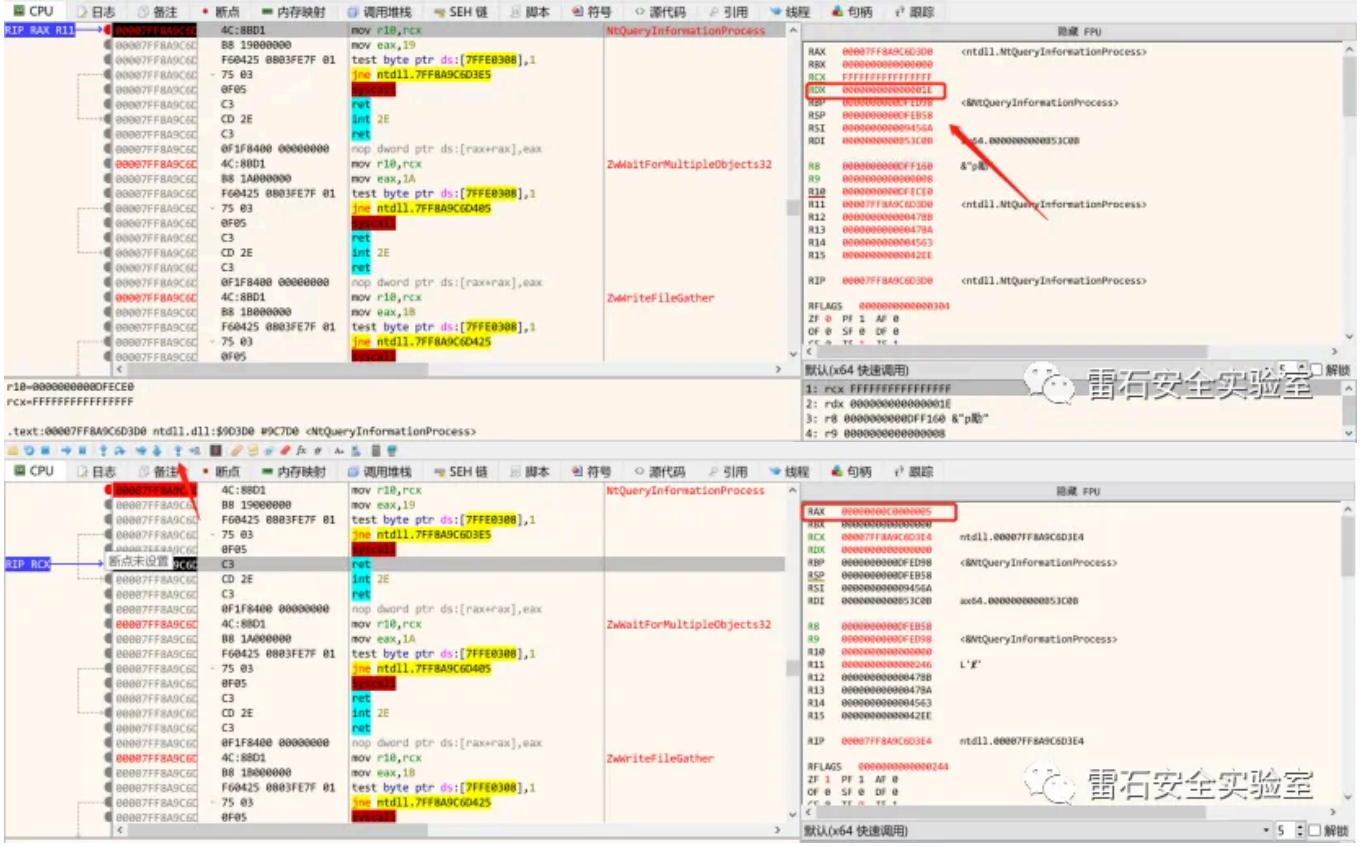
NtQueryInformationProcess：rdx == 0x07 把r8 置为nullptr（0）



再运行，

NtQueryInformationProcess：rdx ==0x1E 把r8 置为nullptr（0）执行到返回值

ret返回 并将RAX返回值C0000353

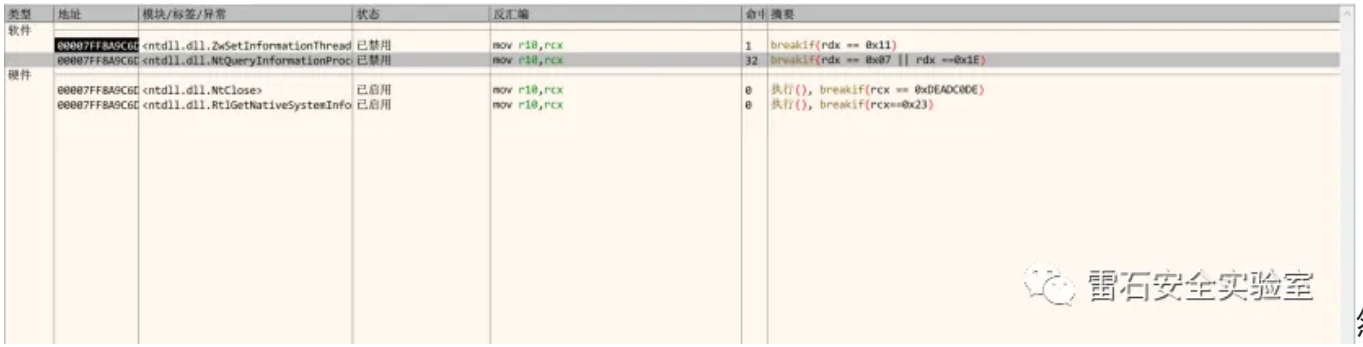


在运行，NtSetInformationThread：rdx == 0x11 将rdx置为3后，禁用这两个断点。

在这两个函数下硬件断点

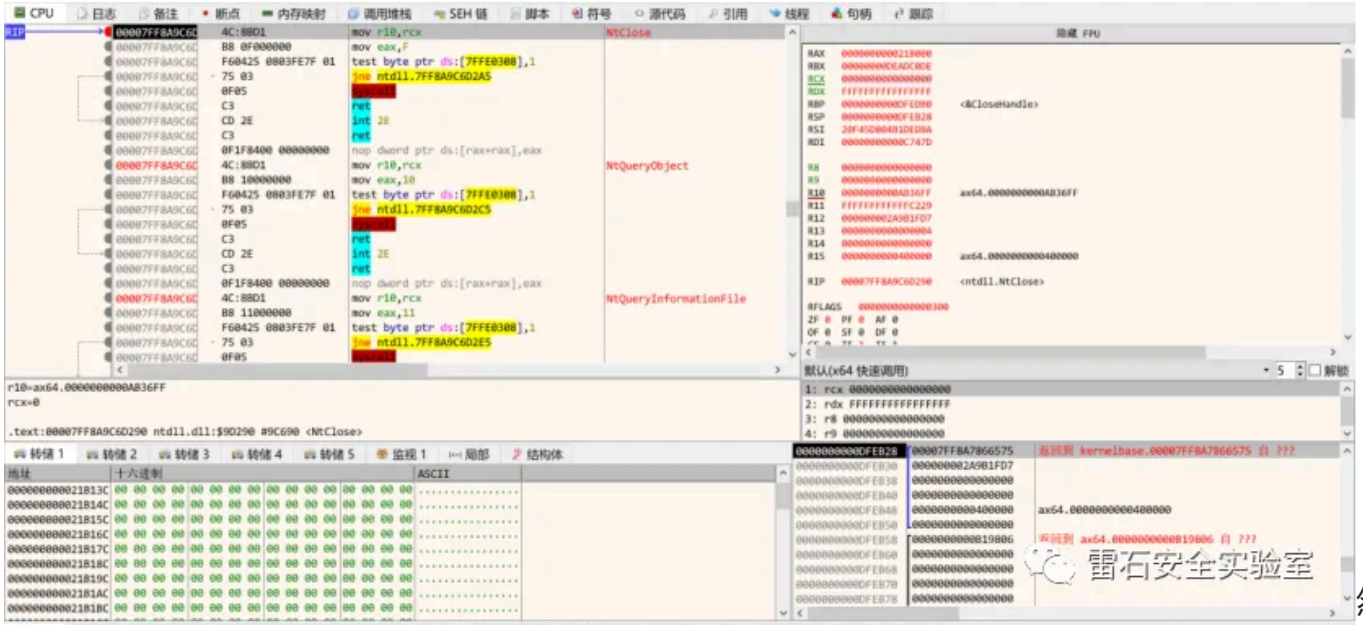
ntclose 硬件断点：NtClose函数在释放无效句柄时。如果没有被调试，那么函数返回FALSE。如果处于调试状态则会抛出异常C0000008H 并设置条件 rcx == 0xDEADC0DE

NtQuerySystemInformation硬件断点：被 ntdll.dll 导出，当第一个参数传入 0x23 （SystemInterruptInformation） 时，会返回一个SYSTEM_KERNEL_DEBUGGER_INFORMATION 结构，里面的成员KdKdDebuggerEnable 和 KdDebuggerNotPresent 标志系统是否启用内核调试。并设置条件 rcx == 0x23

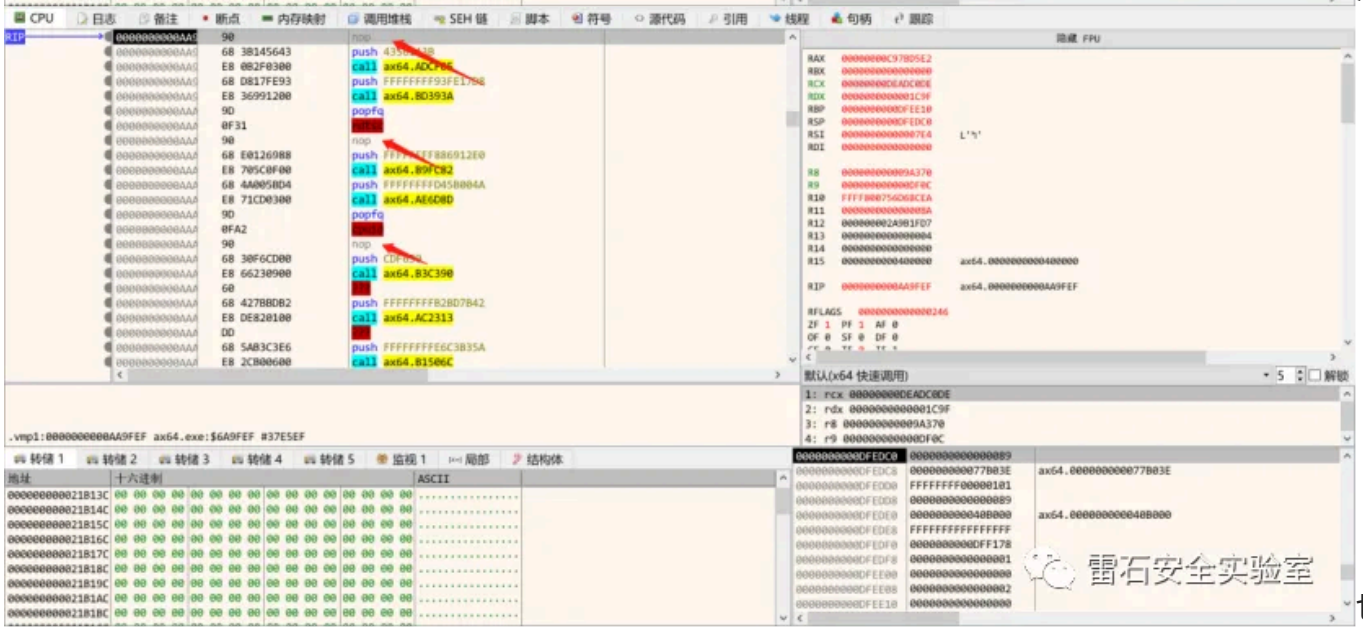


然后运行如果执行到NtQuerySystemInformation 且rcx==0x23 执行到返回 然后将RAX变为status_unsuccessful（0xC0000001）

然后运行到ntclose 并将RCX设置为0（flase）



然后清除（禁用）所有断点然后运行



也是成功的绕过了反调试，shift+F9运行在进行调试中就不会有任何报错。

以上就是此次关于反调试技术的一些想法分享，下一篇晚点见~